Supervyse OPF Intel MCTP Porting Guide

Revision 1.2

12/2024

Copyright (c) 2024, All Rights Reserved. Insyde Software.

No part of this publication may be reproduced, transmitted, transcribed, stored in a retrieval system, or translated into any language or computer language, in any form, or by any means, electronic, mechanical, magnetic, optical, chemical, manual or otherwise, without the prior written permission of Insyde Software Corp.

Disclaimer

Insyde Software provides this document and the programs "as is" without warranty of any kind, either expressed or implied, including, but not limited to, the implied warranties of merchantability or fitness for a particular purpose.

This document could contain technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in future revisions of this document. Insyde Software is under no obligation to notify any person of the changes.

The following trademarks are used in this document:

Insyde and InsydeH2O are registered trademarks or trademarks of Insyde Software. All other trademarks or trade names are property of their respective holders.

Contents

Supervyse OPF Intel MCTP Porting Guide 5
1. Introduction 5
2. MCTP over PCIe 6
2.1 Setup with Insyde BIOS 6
2.2 Kernel DTS configuration 6
2.3 entity-manager configuration 7
2.4 Verification and debug 8
3. MCTP over SMBus 10
3.1 entity-manager configuration 10
3.2 I2C MUX configuration 13
3.2.1 Purpose 13
3.2.2 Configuration with entity-manager 13
3.3 Verification and debug 15

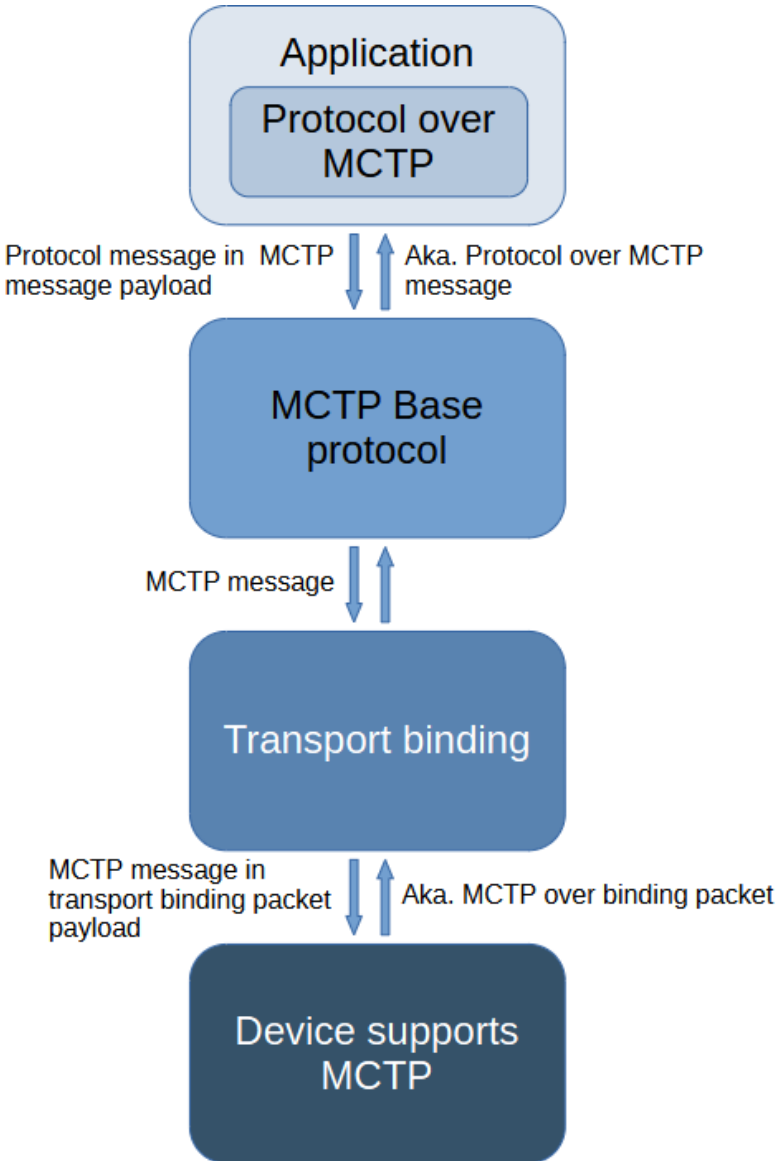
Revision History

Revision Number	Date	Author	Reviewer	Summary
1.0	2024-06-07	Jim Pai	Will Tsai	RV22.11 Release Document
1.1	2024-06-07	Will Tsai	Jim Pai	Moved from user guide
1.2	2024-12-26	Will Tsai	Jim Pai	Refine the document

Supervyse OPF Intel MCTP Porting Guide

1. Introduction

Management Component Transport Protocol (MCTP) is a protocol designed by the Distributed Management Task Force (DMTF). MCTP is intended to be used for intercommunication between BMC and managed devices. MCTP can potentially be used on different bus types, like PCIe or SMBus



The MCTP message will pass through layers during transmission from the sender to the receiver:

- Protocol over MCTP

A protocol that is designed to use MCTP to transmit protocol messages. The protocol message will be encapsulated in the payload of an MCTP message. For example, NC-SI over MCTP is a protocol that transmits NC-SI commands using MCTP.

- MCTP Base protocol

  The protocol is responsible to route MCTP-formatted messages from source to destination.

- Transport binding

  Encapsulate MCTP messages into the packet format that can be transmitted on a specific physical interface. Transport binding also defines how to transmit these packets over the interface. For example, PCIe transport binding defines how to transmit MCTP over PCIe packets on PCIe bus.

MCTP constructs a network that enables devices within the network to communicate with each other. In the MCTP network, all devices are MCTP endpoints logically addressed by MCTP endpoint IDs (EIDs), so there is no difference when sending messages to an endpoint over PCIe, SMBus, or other physical interfaces.

This guide introduces the configurations to enable MCTP over PCIe and MCTP over SMBus feature.

2. MCTP over PCIe

2.1 Setup with Insyde BIOS

- Eagle Steam platforms
  - ☐ Go to **Advance -> Server ME Configuration**
  - ☐ Check **MctpInfrastructure** in Server ME Firmware Features List
  - ☐ Check **MctpProxy NOT** in Server ME Firmware Features List
- Birch Stream platforms
  - ☐ Go to **Advanced -> Socket Configuration -> Uncore Configuration -> Uncore General Configuration**
  - ☐ Check **MCTP Bus Owner Selection** selects **CPU as Bus Owner**

2.2 Kernel DTS configuration

- Add kernel DTS configuration for Aspeed MCTP PCIe driver
  - ☐ <SDK>/meta-insyde/meta-intel/meta-<PLATFORM NAME>/recipes-platform/linux-dts/linux-aspeed/insyde_overlay/arch/arm/boot/dts/aspeed-g6.dtsi
  - ☐ Example configuration file for reference
    - <SDK>/meta-insyde/meta-intel/meta-archercity/recipes-platform/linux-dts/linux-aspeed/insyde_overlay/arch/arm/boot/dts/aspeed-g6.dtsi

```
pcieh: pcieh@1e6ed000 {
    compatible = "aspeed,ast2600-pcieh", "syscon";
    reg = <0x1e6ed000 0x100>;
};

mctp: mctp@1e6e8000 {
    compatible = "aspeed,ast2600-mctp";
    reg = <0x1e6e8000 0x1000>;
    interrupts-extended = <&gic GIC_SPI 26 IRQ_TYPE_LEVEL_HIGH>,
                <&scu_ic0 ASPEED_AST2600_SCU_IC0_PCIE_PERST_LO_TO_HI>;
    resets = <&syscon ASPEED_RESET_DEV_MCTP>;
```

```
    aspeed,pcieh = <&pcieh>;
    status = "disabled";
};
```

- `pcieh` identifies the PCIe host controller device node located at memory address 0x1e6ed000. `pcieh` is referenced by MCTP device node and will be used to read PCIe host controller status by MCTP driver
  - `compatible` specifies which device driver is compatible with this hardware
  - `reg` specifies the memory-mapped address range for the device is 0x100 bytes from 0x1e6ed000
- `mctp` identifies the MCTP controller device node located at memory address 0x1e6e8000
  - `compatible` specifies which device driver is compatible with this hardware
  - `reg` specifies the memory-mapped address range for the device is 0x1000 bytes from 0x1e6e8000
  - `interrupts-extended` defines
    - the interrupt from Generic Interrupt Controller (GIC) for MCTP controller status change processing
    - the interrupt from System Control Unit (SCU) for PCIe PERST signal processing
  - `resets` specifies the reset is triggered from SCU for MCTP device
  - `aspeed,pcieh` is the label used by MCTP driver to reference `pcieh` device node to read the register
  - `status` indicates the initial status of the device

- Enable MCTP device
  - ☐ <SDK>/meta-insyde/meta-intel/meta-<PLATFORM NAME>/recipes-platform/linux-dts/linux-aspeed/insyde_overlay/arch/arm/boot/dts/aspeed-bmc-intel-ast2600.dts
  - ☐ Example configuration file for reference
    - <SDK>/meta-insyde/meta-intel/meta-archercity/recipes-platform/linux-dts/linux-aspeed/insyde_overlay/arch/arm/boot/dts/aspeed-bmc-intel-ast2600.dts

```
&mctp {
    status = "okay";
};
```

2.3 entity-manager configuration

- To launch a MCTP over PCIe services, MctpConfiguration with type MctpPCIe must be added to entity-manager configuration json file
  - ☐ <SDK>/meta-insyde/meta-intel/meta-<PLATFORM NAME>/recipes-platform/configuration/entity-manager/insyde_overlay/configurations/plat-<CRB NAME>/<CRB NAME>-Baseboard.json
  - ☐ Below is example MctpConfiguration and definition of fields

```
{
        "BDF": 0,
        "BindingType": "MctpPCIe",
        "DefaultEID": 0,
        "Index": 1,
        "Name": "MCTP PCIe",
        "PhysicalMediumID": "Pcie2",
        "ReqRetryCount": 2,
        "ReqToRespTimeMs": 120,
```

```
        "GetRoutingInterval": 5,
        "Role": "Endpoint",
        "Type": "MctpConfiguration"
}
```

MCTP over PCIe MctpConfiguration definition of fields				
Name	**Description**	**Type**	**Customize[1]**	**Default[2]**
BDF	PCIe BDF address of BMC. Will be overwritten by MCTP service initialization	uint16	N	-
BindingType	Transport binding type	string	N	-
DefaultEID	Will be overwritten with the EID assigned by bus owner	uint8	N	-
Index	No function	int32	N	1
Name	Suffix of MCTP D-Bus service name	string	N	-
PhysicalMediumID	MCTP physical media identifier	string	N	-
ReqRetryCount	Request retry count for control message	uint8	Y	-
ReqToRespTimeMs	Tx retry delay in milliseconds for control message	uint32	Y	-
GetRoutingInterval	Interval in seconds to query routing table from bus owner	uint8	Y	-
Role	Role of MCTP endpoint	string	N	-
Type	Configuration type name for entity-manager	string	N	-

Note:
1. Customize 'Y' means the field can be changed to meet platform requirement.
2. Default means the field will be filled with a default value if it is not found in MctpConfiguration. Default value "-" means the field is mandatory.

2.4 Verification and debug

- Verify MCTP driver is probed successfully

  1. There is no error message related to "aspeed-mctp" in kernel log

  2. Device node /dev/aspeed-mctp is created

  Check configuration in section 2.2 if the verification failed

- Verify MCTP service is started successfully

  ☐ Find MCTP service name first

```
# systemctl list-units --all | grep MCTP_PCIe
  xyz.openbmc_project.mctpd@BNC_Baseboard_2fMCTP_PCIe.service
```

  - In the example, service name is `xyz.openbmc_project.mctpd@BNC_Baseboard_2fMCTP_PCIe.service`

  ☐ Service status should be "active (running)"

```
# systemctl status \
xyz.openbmc_project.mctpd@BNC_Baseboard_2fMCTP_PCIe.service
```

- ☐ If the verification failed
  1. Check configuration section 2.3
  2. Check error message in journal log

```
# journalctl -u \
xyz.openbmc_project.mctpd@BNC_Baseboard_2fMCTP_PCIe.service
```

- Verify MCTP D-Bus service is started successfully
  - ☐ Following command should be executed without error

```
# busctl tree xyz.openbmc_project.MCTP_PCIe
```

- ☐ If the verification failed
  1. Check configuration section 2.3
  2. Check error message in journal log

```
# journalctl -u \
xyz.openbmc_project.mctpd@BNC_Baseboard_2fMCTP_PCIe.service
```

- Verify MCTP service can find any endpoint
  - ☐ Check MCTP service D-Bus tree

```
# busctl tree xyz.openbmc_project.MCTP_PCIe
`-/xyz
  `-/xyz/openbmc_project
    `-/xyz/openbmc_project/mctp
      `-/xyz/openbmc_project/mctp/device
        |-/xyz/openbmc_project/mctp/device/10
        |-/xyz/openbmc_project/mctp/device/26
        |-/xyz/openbmc_project/mctp/device/27
        |-/xyz/openbmc_project/mctp/device/28
        |-/xyz/openbmc_project/mctp/device/29
        `-/xyz/openbmc_project/mctp/device/8
```

- ☐ MCTP is working if users can see the D-Bus objects like /xyz/openbmc_project/mctp/device/X

- ☐ If MCTP is not working, stop MCTP service, and then run MCTP service manually with detailed debug log enabled

```
# systemctl stop xyz.openbmc_project.mctpd@BNC_Baseboard_2fMCTP_PCIe.service
# export MCTP_TRACES=1; /usr/bin/mctpd -b BNC_Baseboard_2fMCTP_PCIe
```

3. MCTP over SMBus

3.1 entity-manager configuration

- To launch MCTP over SMBus services, MctpConfiguration with type MctpSMBus must be added to entity-manager configuration json file
  - ☐ <SDK>/meta-insyde/meta-intel/meta-<PLATFORM NAME>/recipes-platform/configuration/entity-manager/insyde_overlay/configurations/plat-<CRB NAME>/<CRB NAME>-Baseboard.json
  - ☐ Below is example MctpConfiguration and definition of fields for OPF G3

```
{
      "ARPOwnerSupport": false,
      "BMCReceiverAddress": 32,
      "BindingType": "MctpSMBus",
      "Bus": "/dev/i2c-8",
      "DefaultEID": 128,
      "EIDPool": [
            129,
            130,
            131,
            132,
            133,
            134,
            135,
            136,
            137,
            138,
            139,
            140
      ],
      "Index": 0,
      "Name": "MCTP SMBus PCIe slot",
      "PhysicalMediumID": "Smbus3OrI2c400khzCompatible",
      "ReqRetryCount": 2,
      "ReqToRespTimeMs": 100,
      "Role": "BusOwner",
      "TopMostBusOwner": false,
      "OwnEidPool": true,
      "Type": "MctpConfiguration"
}
```

MCTP over SMBus MctpConfiguration definition of fields

Name	Description	Type	Customize[1]	Default[2]
ARPOwnerSupport	No function	bool	N	false
BMCReceiverAddress	BMC SMBus 8-bits slave address	uint8	Y	-
BindingType	Transport binding type	string	N	-
Bus	I2C device node path to scan	string	Y	-
DefaultEID	Default EID of BMC	uint8	Y	-
EIDPool	EID pool of SMBus bus owner. Should be consecutive values. Should exclude DefaultEID and special EIDs	uint8 array	Y	-
Index	No function	int32	N	0
Name	Suffix of MCTP D-Bus service name	string	N	-
PhysicalMediumID	MCTP physical media identifier	string	N	-
ReqRetryCount	Request retry count for control message	uint8	Y	-
ReqToRespTimeMs	Tx retry delay in milliseconds for control message	uint32	Y	-
Role	Role of MCTP endpoint	string	N	-
TopMostBusOwner	No function	bool	N	false
OwnEidPool	No function	bool	N	true
Type	Configuration type name for entity-manager	string	N	-

Note:
1. Customize 'Y' means the field can be changed to meet platform requirement.
2. Default means the field will be filled with a default value if it is not found in MctpConfiguration. Default value "-" means the field is mandatory.

- Below is default MctpConfiguration values and definition of fields for OPF G4

```
{
	"ARPOwnerSupport": false,
	"AllowedBuses": [
		"xyz.openbmc_project.MCTP_SMBus_HSBP"
	],
	"BMCReceiverAddress": 18,
	"Bus": "/dev/i2c-9",
	"DefaultEID": 8,
	"EIDPool": [
		10,
		11,
		12,
		13,
		14,
		15,
		16,
		17,
		18,
		19,
		20
	],
	"IgnoredEndpointTargetAddress": [
		80,
```

```
            81,
            82,
            83,
            84,
            85,
            86,
            87
        ],
        "Index": 1,
        "Name": "MCTP SMBus PCIe slot",
        "OwnEidPool": true,
        "PhysicalMediumID": "I2cCompatible",
        "ReqRetryCount": 2,
        "ReqToRespTimeMs": 300,
        "Role": "BusOwner",
        "SupportsBridge": true,
        "TopMostBusOwner": false,
        "TransportBindingType": "MctpSMBus",
        "Type": "MctpConfiguration"
}
```

MCTP over SMBus MctpConfiguration definition of fields				
Name	**Description**	**Type**	**Customize[1]**	**Default[2]**
ARPOwnerSupport	No function	bool	N	false
AllowedBuses	The downstream MCTP services are allowed to pass MCTP messages to	string array	Y	empty array
BMCReceiverAddress	BMC SMBus 8-bits slave address	uint8	Y	-
Bus	I2C device node path to scan	string	Y	-
DefaultEID	Default EID of BMC	uint8	Y	-
EIDPool	EID pool of SMBus bus owner. Should be consecutive values. Should exclude DefaultEID and special EIDs	uint8 array	Y	-
IgnoredEndpointTargetAddress	The I2C addresses of devices will be skipped when scanning	uint8 array	Y	empty array
Index	No function	int32	N	2
Name	Suffix of MCTP D-Bus service name	string	N	-
OwnEidPool	No function	true	N	true
PhysicalMediumID	MCTP physical media identifier	string	N	-
ReqRetryCount	Request retry count for control message	uint8	Y	-
ReqToRespTimeMs	Tx retry delay in milliseconds for control message	uint32	Y	-
Role	Role of MCTP endpoint	string	N	-
SupportsBridge	Support bridging. Will not be read from MctpConfiguration	bool	Y	false
TopMostBusOwner	No function	bool	N	false
TransportBindingType	Transport binding type	uint8	N	-

Type	Configuration type name for entity-manager	string	N	-

Note:
1. Customize 'Y' means the field can be changed to meet platform requirement.
2. Default means the field will be filled to the default value if it is not found in MctpConfiguration. The fields without Default value are mandatory.

3.2 I2C MUX configuration

3.2.1 Purpose

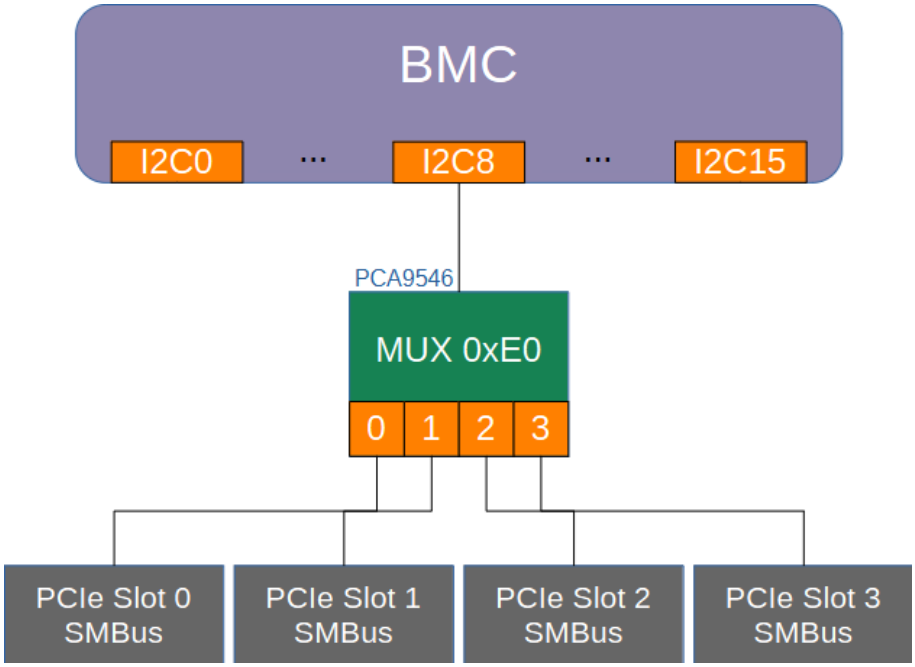
- For the MCTP service to read/write data from/to the devices on the MUX channels, each MUX channel must be associated with a device node in the form of /dev/i2c-X, so that the MUX will switch to the corresponding channel when the device node is accessed (e.g., via an ioctl call using the file descriptor opened from the device node)
  - ☐ The creation of channel device nodes and channel switching are managed by kernel I2C framework mechanism
  - ☐ Simply configure the kernel to achieve the purpose

3.2.2 Configuration with entity-manager

- There are other ways to configure kernel for I2C MUXs, this section introduces a method to configure kernel with entity-manager for PCA9546 I2C MUXs

  - ☐ For other ways to configure, please refer to the kernel document about I2C MUX

- Use following I2C topology as the example to configure



  - ☐ From the I2C topology we know
    - The MUX PCA9546 is on BMC I2C bus 8
    - The address of the MUX is 0x70/7-bit (0xE0/8-bit)

☐ Add following configuration to entity-manger configuration json file

```
{
        "Address": "0x70",
        "Bus": 8,
        "ChannelNames": [
                "Pcie_Slot_0",
                "Pcie_Slot_1",
                "Pcie_Slot_2",
                "Pcie_Slot_3"
        ],
        "Name": "PCIE Mux",
        "Type": "PCA9546Mux"
}
```

Field	Description
Address	The I2C address of the MUX in 7-bit format
Bus	The BMC I2C bus the MUX is connected to
ChannelNames	The names are provided for ease of identification. The number of names correspond to the number of I2C device nodes to be created. The first name represents channel 0, the second name represents channel 1, and so on
Name	The name of the I2C MUX
Type	Configuration type. Must be PCA9546Mux for PCA9546

☐ Check created paths of device nodes via sysfs

1. Go to the directory of i2c-8

```
~# cd /sys/bus/i2c/devices/i2c-8
```

2. There should be a 0x70 MUX as 8-0070. Go inside it and list its content

```
/sys/bus/i2c/devices/i2c-8# cd 8-0070
/sys/bus/i2c/devices/i2c-8/8-0070# ls -l
total 0
lrwxrwxrwx 1 root root    0 Feb 26  2022 channel-0 -> ../i2c-22
lrwxrwxrwx 1 root root    0 Feb 26  2022 channel-1 -> ../i2c-23
lrwxrwxrwx 1 root root    0 Feb 26  2022 channel-2 -> ../i2c-24
lrwxrwxrwx 1 root root    0 Feb 26  2022 channel-3 -> ../i2c-25
...
```

3. So, we can know
   - channel-0 is /dev/i2c-22
   - channel-1 is /dev/i2c-23
   - channel-2 is /dev/i2c-24
   - channel-3 is /dev/i2c-25
4. If the device nodes are not created, check kernel log for i2c related error messages

3.3 Verification and debug

- Verify MCTP service is started successfully
  - ☐ Find MCTP service name first

```
# systemctl list-units --all | grep MCTP_SMBus_PCIe_slot
  xyz.openbmc_project.mctpd@BNC_Baseboard_2fMCTP_SMBus_PCIe_slot.service
```

    - Service name is `xyz.openbmc_project.mctpd@BNC_Baseboard_2fMCTP_SMBus_PCIe_slot.service`

  - ☐ Service status should be "active (running)"

```
# systemctl status \
xyz.openbmc_project.mctpd@BNC_Baseboard_2fMCTP_SMBus_PCIe_slot.service
```

  - ☐ If the verification failed
    1. Check configuration section 2.3
    2. Check error message in journal log

```
# journalctl -u \
xyz.openbmc_project.mctpd@BNC_Baseboard_2fMCTP_SMBus_PCIe_slot.service
```

- Verify the I2C slave is created for receiving MCTP packets on I2C bus
  - ☐ The I2C slave is created automatically by MCTP service
  - ☐ Suppose "BMCReceiverAddress" is 32 (0x10/7-bit), and "Bus" is /dev/i2c-8 in entity-manager configuration. There should be a file named **slave-mqueue** under /sys/bus/i2c/devices/i2c-X/X-10YY
    - X is 8 for I2C bus 8
    - YY is 10 for 7-bit i2C address 10 in hexadecimal

```
/sys/bus/i2c/devices/i2c-8/8-1010# ls
driver  modalias  name  of_node  slave-mqueue  subsystem  uevent
```

  - ☐ If the verification failed, check kernel log for errors related to i2c

- Verify MCTP D-Bus service is started successfully
  - ☐ Following command should be executed without error

```
# busctl tree xyz.openbmc_project.MCTP_SMBus_PCIe_slot
```

  - ☐ If the verification failed
    1. Check configuration section 2.3

2. Check error message in journal log

```
# journalctl -u \
xyz.openbmc_project.mctpd@BNC_Baseboard_2fMCTP_SMBus_PCIe_slot.service
```

- Verify MCTP service can find any endpoint

  ☐ Check MCTP service D-Bus tree

```
# busctl tree xyz.openbmc_project.MCTP_SMBus_PCIe_slot
`-/xyz
  `-/xyz/openbmc_project
    `-/xyz/openbmc_project/mctp
      `-/xyz/openbmc_project/mctp/device
        |-/xyz/openbmc_project/mctp/device/10
        `-/xyz/openbmc_project/mctp/device/11
```

  ☐ MCTP is working if users can see any D-Bus object like /xyz/openbmc_project/mctp/device/X

  ☐ If MCTP is not working, stop MCTP service, and then run MCTP service manually with detailed debug log enabled

```
# systemctl stop \
xyz.openbmc_project.mctpd@BNC_Baseboard_2fMCTP_SMBus_PCIe_slot.service
# export MCTP_TRACES=1; \
/usr/bin/mctpd -b BNC_Baseboard_2fMCTP_SMBus_PCIe_slot
```